Module: Flask Overview — Micro Web Framework & Notes API

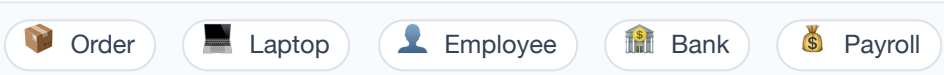
Previous module: Lab Deploy Capstone Debug
Next module: APIs JSON Postman DRF

Module Outcome:

- Stand up a tiny Flask app and map a URL to a function
- Return HTML or JSON without Django's admin/ORM — on purpose
- See when Flask is the right small tool vs when CampusCart stays on Django

What is Flask?

Flask is a lightweight Python **web framework**. You write a small Python file, map URLs to functions, and return HTML or JSON. Unlike Django, Flask does not ship with admin, ORM, or auth — you add what you need.



Where used?
- **Swiggy** — tiny Notes / product JSON APIs for hackathons and microservices
- College apps — quick campus tools without full Django admin
- Startups — Flask/FastAPI-style API roles next to React
- **HDFC Bank** / **Employee Management Portal** — internal microservices that only need a few routes
- Prototypes — prove an endpoint before wiring Django/DRF

Analogy: Django is a furnished apartment (kitchen, bed, wardrobe included). Flask is an empty studio — you bring furniture as required. For a 2-day hackathon API, Flask is faster to start. For a full HR portal with login and admin, Django wins.

Flask vs Django (India job context)

Aspect	Flask	Django
Size	Micro — minimal core	Full-stack — many built-ins
Learning curve	Gentle for first HTTP API	Steeper but teaches structure
ORM	Add SQLAlchemy separately	Django ORM built-in
Admin panel	None by default	Powerful admin
Typical use	Microservices, APIs, prototypes	ERP, e-commerce, CMS
Indian hiring	Startups, API roles	Service companies, product teams

Both appear in job posts. Full-stack Python roles often say "Django/Flask" — knowing Flask proves you understand HTTP without framework magic.

Install Flask

```
cd ~/ksr_python_fullstack
python3 -m venv venv
source venv/bin/activate          # Windows: venv\Scripts\activate
pip install flask
pip freeze > requirements.txt
```

Note:
- Running without an activated venv → wrong Flask version / permission errors. Always activate venv first.
- `debug=True` is for local learning only — never ship it in production (use Gunicorn + env config later).

Concepts

1. Application object

```
from flask import Flask
app = Flask(__name__)   # __name__ helps Flask find templates folder
```

Build a minimal Flask app that serves a campus notes heading on `/`. Run it with `python app.py` and confirm the dev server starts on port 5000.

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "<h1>Campus Notes — Hyderabad</h1>"

if __name__ == "__main__":
    app.run(debug=True)
```

Expected Output: (terminal after `python app.py`)

```
 * Running on http://127.0.0.1:5000
```

Open `/` in the browser → page heading **Campus Notes — Hyderabad**.

Create a Flask app object and print its `name` attribute to see how Flask identifies the app when you run the file as a script.

```
from flask import Flask
app = Flask(__name__)
print(app.name)  # usually __main__ when run as script
```

Expected Output:

```
__main__
```

Remember: Activate the venv before `pip install flask` / `python app.py` — otherwise you hit the wrong interpreter.

2. Routes and view functions

A **route** connects a URL path to a Python function. The function's return value becomes the HTTP response body.

Note:
- Route typo → 404. Check the URL, trailing slash, and method (GET vs POST).

Add two routes — `/` and `/about` — that return simple text responses for home and about pages.

```
from flask import Flask
app = Flask(__name__)

@app.route("/")
def home():
    return "Home"

@app.route("/about")
def about():
    return "About CampusCart Notes API"

if __name__ == "__main__":
    app.run(debug=True)
```

Add a route with a `<name>` path parameter and visit `/user/Priya` in the browser to test the greeting.

```
from flask import Flask
app = Flask(__name__)

@app.route("/user/<name>")
def greet(name):
    return f"Namaste, {name}!"

if __name__ == "__main__":
    app.run(debug=True)
# Visit /user/Priya
```

Expected Output: (browser at `/user/Priya`)

```
Namaste, Priya!
```

Common Error: Hitting `/user/Priya/` when the route is `/user/<name>` (or the reverse) → 404. Match the trailing slash to the `@app.route`.

Interview: Flask vs Django — Flask = micro framework (add pieces); Django = batteries included (admin, ORM, auth). Job posts often list both.

3. HTTP methods

Method	Purpose	Notes API example
GET	Read data	List all notes
POST	Create	Add new note
PUT	Update (full)	Replace note by id
DELETE	Remove	Delete note by id

Create a `/ping` route that accepts both GET and POST and returns which HTTP method the client used.

```
from flask import Flask, request
app = Flask(__name__)

@app.route("/ping", methods=["GET", "POST"])
def ping():
    return f"You used {request.method}"

if __name__ == "__main__":
    app.run(debug=True)
```

Return a JSON array from `/api/methods` that describes when to use GET, POST, and DELETE for a notes API.

```
from flask import Flask, jsonify
app = Flask(__name__)

@app.route("/api/methods")
def methods():
    return jsonify([
        {"method": "GET", "use": "list notes"},
        {"method": "POST", "use": "create note"},
        {"method": "DELETE", "use": "remove note"},
    ])

if __name__ == "__main__":
    app.run(debug=True)
```

4. Templates (Jinja2)

Flask uses **Jinja2** (same family as Django templates). Files live in `templates/`. Use `{{ variable }}` and `{% for %}` loops.

> **Note:**
> - Folder must be named exactly `templates` next to the app — a typo → `TemplateNotFound`.
>
> Pass a notes list from a view function into `render_template` and loop over it in `templates/notes.html` with Jinja2.

```
from flask import Flask, render_template
app = Flask(__name__)

@app.route("/")
def home():
```

```
    notes = [{"title": "Buy coffee", "body": "₹25 filter coffee"}]
    return render_template("notes.html", notes=notes)
```

```
<!-- templates/notes.html -->
<h1>Notes</h1>
<ul>
  {% for n in notes %}
    <li>{{ n.title }} — {{ n.body }}</li>
  {% endfor %}
</ul>
```

Display the total number of notes in the template using Jinja2's `length` filter.

```
<p>Total notes: {{ notes|length }}</p>
```

5. JSON for APIs

Mobile apps and React frontends consume **JSON**, not HTML. Use `jsonify()` to return Python dicts as JSON with correct `Content-Type`.

> **Note:**
> - POST without `Content-Type: application/json` → `request.get_json()` returns `None`. Send the JSON header in Postman/curl.

Expose `/api/notes` as a GET endpoint that returns a JSON list of notes using `jsonify()`.

```
from flask import Flask, jsonify
app = Flask(__name__)

@app.route("/api/notes")
def list_notes():
    return jsonify([{"id": 1, "title": "Flask lab", "body": "Done today"}])

if __name__ == "__main__":
    app.run(debug=True)
```

Implement POST on `/api/notes` that requires a `title` field, returns 201 on success, and 400 when the title is missing.

```
from flask import Flask, jsonify, request
app = Flask(__name__)
notes = []

@app.route("/api/notes", methods=["POST"])
def create_note():
    data = request.get_json() or {}
    if "title" not in data:
        return jsonify({"error": "title required"}), 400
    note = {"id": len(notes) + 1, "title": data["title"]}
    notes.append(note)
    return jsonify(note), 201

if __name__ == "__main__":
    app.run(debug=True)
```

Step-by-Step Annotated Code

Step 1 — Hello Flask

Create `flask_demo/app.py`:

```
from flask import Flask

app = Flask(__name__)          # Create application instance

@app.route('/')                # Map URL "/" to function below
def home():
    return '<h1>Hello from Flask!</h1>'   # Return HTML string

if __name__ == '__main__':
    app.run(debug=True)        # debug=True: auto-reload + better errors (dev only)
```

Run: `python app.py` → open `http://127.0.0.1:5000`

Step 2 — Route with parameter

```
@app.route('/user/<name>')     # <name> captured from URL
def greet(name):
    return f'Hello, {name}!'
```

Visit `/user/Priya` → `Hello, Priya!`

Steps 3–5 (templates, JSON, Notes API) are included in full below — follow them in order on your machine.

Step 3 — Complete Notes API (in-memory)

```
from flask import Flask, jsonify, request, render_template

app = Flask(__name__)

# In-memory store — resets when server restarts (fine for learning)
notes = []
next_id = 1

@app.route('/')
def home():
    return render_template('notes.html', notes=notes, count=len(notes))

@app.route('/api/notes', methods=['GET'])
def list_notes():
    return jsonify(notes)

@app.route('/api/notes', methods=['POST'])
def create_note():
    global next_id
    data = request.get_json()              # Parse JSON body from client
    if not data or 'title' not in data:
        return jsonify({'error': 'title required'}), 400
    note = {'id': next_id, 'title': data['title'], 'body': data.get('body', '')}
    next_id += 1
    notes.append(note)
    return jsonify(note), 201              # 201 Created

@app.route('/api/notes/<int:note_id>', methods=['GET'])
```

```
def get_note(note_id):
    for note in notes:
        if note['id'] == note_id:
            return jsonify(note)
    return jsonify({'error': 'not found'}), 404

@app.route('/api/notes/<int:note_id>', methods=['PUT'])
def update_note(note_id):
    data = request.get_json()
    for note in notes:
        if note['id'] == note_id:
            note['title'] = data.get('title', note['title'])
            note['body'] = data.get('body', note['body'])
            return jsonify(note)
    return jsonify({'error': 'not found'}), 404

@app.route('/api/notes/<int:note_id>', methods=['DELETE'])
def delete_note(note_id):
    global notes
    for i, note in enumerate(notes):
        if note['id'] == note_id:
            notes.pop(i)
            return '', 204                    # 204 No Content
    return jsonify({'error': 'not found'}), 404

if __name__ == '__main__':
    app.run(debug=True)
```

Test with curl:

```
curl -X POST http://127.0.0.1:5000/api/notes \
  -H "Content-Type: application/json" \
  -d '{"title":"Flask lab","body":"Done today"}'

curl http://127.0.0.1:5000/api/notes
```

Note:
- In-memory notes vanish on restart — fine for learning; use a DB for real apps.
- Modifying a list while iterating can skip items — prefer a list comprehension or `pop` by index carefully (as in `delete_note` above).

Practice Exercises

Basic

1. Add route `/about` returning plain text about your training batch.
   Model answer:

```
@app.get("/about")
def about():
    return "KSR Python Full Stack batch — learning Flask APIs."
```

2. Add route `/square/<int:n>` returning JSON `{"input": 5, "result": 25}`.
   Model answer:

```
from flask import jsonify

@app.get("/square/<int:n>")
def square(n):
    return jsonify({"input": n, "result": n * n})
```

3. Create template `base.html` with a footer "KSR Python Full Stack 2026".
   Model answer:

```
<!DOCTYPE html>
<html lang="en">
<body>
  {% block content %}{% endblock %}
  <footer>KSR Python Full Stack 2026</footer>
</body>
</html>
```

Intermediate

1. Add query param filter: `GET /api/notes?search=flask` filters by title substring.
   Model answer:

```
@app.get("/api/notes")
def list_notes():
    q = (request.args.get("search") or "").lower()
    notes = [n for n in NOTES if q in n["title"].lower()] if q else NOTES
    return jsonify(notes)
```

2. Validate POST: reject empty title with 400 and error message.
   Model answer:

```
data = request.get_json(silent=True) or {}
title = (data.get("title") or "").strip()
if not title:
    return jsonify({"error": "title required"}), 400
```

3. Add `GET /api/stats` returning `{"total": N, "latest_id": M}`.
   Model answer:

```
@app.get("/api/stats")
def stats():
    latest = NOTES[-1]["id"] if NOTES else None
    return jsonify({"total": len(NOTES), "latest_id": latest})
```

Interview-style

1. Explain difference between `return dict` and `return jsonify(dict)` in Flask.
   Model answer:
   Modern Flask auto-converts a returned `dict` to JSON. `jsonify` explicitly builds a JSON response (and can set status/headers). Both can work; `jsonify` is clearer when you need custom status codes.

2. How would you connect Flask to MySQL instead of in-memory list? (Hint: SQLAlchemy or raw connector — conceptual answer OK.)
   Model answer:
   Use Flask-SQLAlchemy (or mysql-connector) with a connection URL from env, define models/tables, and replace list ops with queries/commits. Keep DB credentials out of source.
3. Why is Flask called "WSGI" application? (High level: Python web server interface standard.)
   Model answer:
   WSGI is the Python standard interface between web servers and Python apps. Flask's `app` is a WSGI callable that servers like gunicorn invoke to handle HTTP requests.

Mini-Project (Assignment): Personal Notes API + HTML Dashboard

Lab kit (build here): 15_Lab_Assets/25_flask_notes_api/

Goal: Flask app with HTML list page and JSON API for CRUD on notes.

What to build

- At least 4 API endpoints (list, create, get one, delete)
- One Jinja template showing notes in a table
- `requirements.txt` with Flask pinned
- README with curl examples or Postman screenshot

Stretch

- Add simple CSS in `static/style.css` and link via `url_for('static', filename='style.css')`.
- Add PUT update endpoint and show edited titles in the HTML table.

Done when

- [ ] Lab kit Notes JSON API ticket completed
- [ ] GET/POST list+create work (curl or Postman)
- [ ] Jinja page lists notes
- [ ] README includes at least two curl examples

Interview Q&A

Shared with students after concepts and the mini-project. Use for review and interview prep — questions and model answers are included.

1. If your company already uses Django for the main site, why might a team still add a Flask microservice?
2. Is in-memory storage ever acceptable in production? When?
3. how is a Flask route similar to a function you call in Excel VBA or SAP script — input URL, output response?
4. Should Flask replace Django in our course capstone? Why or why not?
5. What breaks if two users POST notes at the same time with in-memory list? (Intro to concurrency — no deep dive required.)

Q1. What is Flask?
A: Lightweight Python WSGI web framework for building web apps and APIs with minimal boilerplate.

Q2. Flask vs Django — when choose Flask?

A: Small APIs, microservices, prototypes, or when you want full control over components; faster initial setup for simple services.

Q3. What is a route in Flask?

A: URL pattern mapped to a view function via `@app.route`, handling HTTP methods and returning response.

Q4. How do you send JSON from Flask?

A: Return `jsonify(data)` or a dict (Flask 2.2+ can jsonify dicts automatically); set appropriate status code as second return value tuple.

Q5. Is Flask suitable for production?

A: Yes with production WSGI server (Gunicorn, uWSGI), debug off, env-based config, and proper database — not the built-in dev server alone.
